

Voice-Based Concept Understanding Analyser (VBCUA)

AI-ML & Generative AI Track — Project Documentation

Submitted By	Thiramsetti Mercy
Email	mercytiramsetti@gmail.com
Project Name	Voice-Based Concept Understanding Analyser (VBCUA)
Date	04 July 2026

This document consolidates all project-lifecycle deliverables — Brainstorming, Requirement Analysis, Design, Planning, Development, Testing, Documentation, and Demonstration — for the VBCUA project.

1.1 Brainstorming & Idea Prioritization

Completed: 02 July 2026

Date	04 July 2026
Submitted By	Thiramsetti Mercy
Project Name	Voice-Based Concept Understanding Analyser (VBCUA)

Step 1: Brainstorm and Idea Listing

S.No	Idea / Suggestion	Category	Group No.
1	Score spoken answers using simple keyword matching against a model answer.	NLP	1
2	Use sentence-embedding semantic similarity instead of keyword matching, to judge true conceptual understanding rather than exact wording.	NLP / ML	2
3	Add audio-based delivery cues — pauses, filler words, voice energy — as confidence/fluency indicators alongside content accuracy.	Audio / Signal Processing	3
4	Build a full multi-user web platform with login, history, and a trainer dashboard.	Product Scope	4
5	Build a lightweight, single-user, fully offline Streamlit app that needs no paid API keys or hosting.	Product Scope	4
6	Generate a downloadable PDF report per attempt so learners have a shareable record of their evaluation.	Reporting	5

Step 2: Idea Prioritization

Group No.	Final Idea	Feasibility	Importance	Priority	Selected
1	Keyword-matching scoring	High	Low	Low	No — superseded by semantic similarity
2	Sentence-BERT semantic similarity scoring	High	High	High	Yes
3	Audio delivery-quality features (pauses, fillers, energy)	Medium	High	High	Yes
4	Lightweight offline single-user app (vs. full multi-user platform)	High	High	High	Yes
5	Downloadable PDF report	High	Medium	Medium	Yes

1.2 Define Problem Statements

Completed: 02 July 2026

PS	I am	I'm trying to	But	Because	Which makes me feel
PS-1	A student/learner preparing to explain a technical concept verbally (e.g. for an interview or viva).	Verify whether I actually understand the concept, not just memorized its definition.	I have no objective, immediate way to measure the quality of my own spoken explanation.	Self-assessment is subjective, and a human mentor or evaluator isn't always available on demand.	Uncertain about my own preparedness going into a real evaluation.
PS-2	An educator or trainer teaching technical concepts to a batch of students.	Quickly gauge how well each student can articulate a concept out loud.	Manually listening to and grading every student's spoken explanation is time-consuming.	Class sizes are large and verbal understanding is inherently qualitative to assess.	Overwhelmed, and unable to give every student timely, consistent feedback.

1.3 Empathy Map

Completed: 02 July 2026

Persona: Learner preparing for a technical interview or viva

SAYS

- "I think I understand ML, but I'm not sure I can explain it clearly."
- "I keep saying 'um' and 'like' when I try to explain this out loud."
- "I wish someone could just tell me if my answer was good enough."

THINKS

- "Am I just repeating a memorized definition instead of really understanding this?"
- "What would an interviewer actually think of my answer?"
- "I don't want to sound like I'm reading off a textbook."

DOES

- Records themselves practicing spoken answers alone.
- Rehearses explanations out loud before an interview or viva.
- Informally compares their explanation against textbook definitions.

FEELS

- Anxious about being evaluated on the spot.
- Frustrated at not having an objective way to check progress.
- Relieved when they get clear, immediate feedback.

2.1 Customer Journey Map

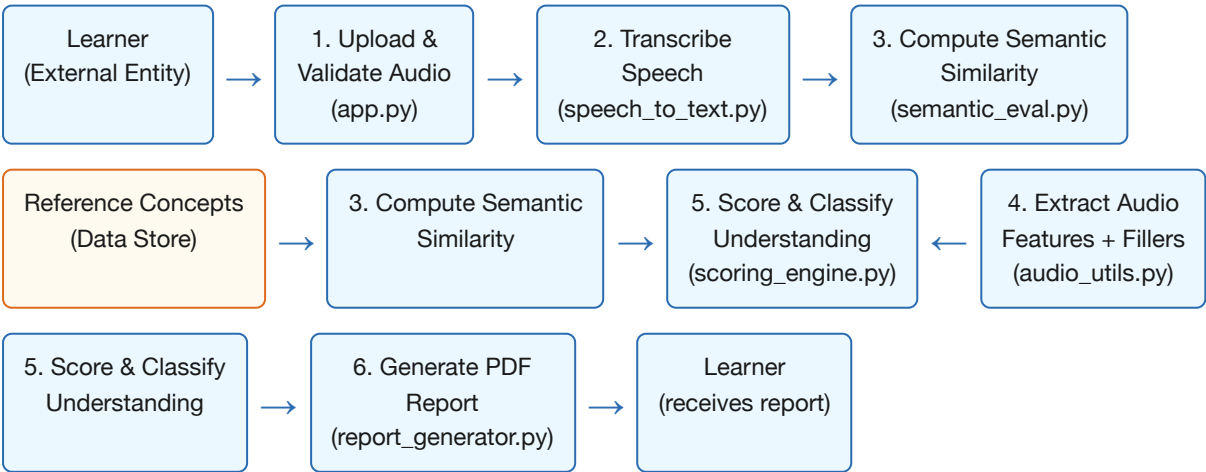
Completed: 02 July 2026

Phase of Journey	Stage 1: Discover & Upload	Stage 2: Analysis	Stage 3: Review & Improve
Actions	Opens the VBCUA app; selects a concept from the dropdown; records or uploads a WAV/MP3 explanation.	Clicks "Analyze Concept Understanding"; waits while transcription, similarity scoring, and audio-feature extraction run.	Reviews transcript, score, classification, and metrics; downloads PDF report; re-records if the score is low.
Touchpoint	Streamlit upload widget, concept selector, reference-text panel.	Spinner/progress indicator; backend pipeline (Whisper, Sentence-BERT, Librosa).	Results panel (score, level, metrics), PDF download button.
Customer Thought	"Did I pick the right concept? Will it accept my file format?"	"Is this actually processing my exact recording?"	"Is my score fair? What should I fix in my explanation?"
Customer Feeling	Curious, slightly unsure about audio format compatibility.	Anticipation, mild suspense while waiting for results.	Validated if the score is high; motivated to improve if it's low.
Process Ownership	End user (self-service)	System (fully automated pipeline, no human in the loop)	End user (self-service)
Opportunities	Add in-browser microphone recording, not just file upload.	Show granular progress ("Transcribing... Scoring...") instead of one spinner.	Let the user compare the current attempt against previous ones to track improvement.

2.2 Data Flow Diagram

Completed: 02 July 2026

VBCUA's data flow for a single analysis request:



Data store used: predefined *Reference Concepts* (8 topics, defined in `reference_concepts.py`). All other state is transient `session_state/temp` files — there is no persistent database in the current version.

2.3 Solution Requirements

Completed: 02 July 2026

Functional Requirements (FR)

S.No	Category	Requirement Description	Priority
1	Authentication	Not implemented — VBCUA is a single-user, local/offline tool with no login or identity verification in this version.	Low
2	Authorization levels	Not applicable — a single implicit user role; no differentiated access levels exist.	Low
3	External interfaces	Uses OpenAI Whisper (base, local) and Sentence-BERT (all-MiniLM-L6-v2, local) as embedded ML libraries — no external network APIs are called at runtime.	High
4	Transactions processing	Each analysis is a single, self-contained request-response cycle: audio in → transcript + score out; no persisted transaction log.	High
5	Reporting	System generates a downloadable PDF report containing the transcript, waveform image, metrics table, and evaluation summary via ReportLab.	High
6	Business rules	Score = weighted sum of semantic similarity (50 pts), filler-word ratio (20 pts), pause ratio (15 pts), RMS energy (15 pts). Classification: ≥80 Strong, 50–79 Moderate, <50 Poor Understanding.	High
7	Compliance to laws / regulations	No PII is collected. Audio is processed in-memory/temp files for the active session only and is not persisted long-term.	Medium
8	Other	Supports WAV and MP3 formats, up to Streamlit's default 200MB upload limit.	Medium

Non-Functional Requirements (NFR)

S.No	Category	Requirement Description	Target / Acceptance Criteria
1	Performance & Speed	Transcription and scoring should complete quickly on standard hardware using the CPU-only Whisper base model.	<60 seconds total pipeline time for a ~30-second clip on a standard laptop CPU
2	Scalability	Not designed for concurrent multi-user load; one Streamlit session per running instance.	Reliable for 1 concurrent user per instance (current scope)
3	Security & Data Privacy	No audio or transcript data is persisted outside the active session; runs fully offline after the initial model download.	Zero external network calls during inference
4	Reliability & Availability	App must not crash on corrupted/unsupported files; validated via a SoundFile read-check before the pipeline runs, with graceful <code>st.error</code> messaging.	Handles 100% of tested invalid-upload cases without crashing
5	Usability & Accessibility	Simple two-column dark-themed layout, clear score/classification display, one-click PDF download.	A first-time user completes an analysis without external instructions

6	Other (Maintainability)	Codebase split into 6 single-responsibility modules for easy modification and testing.	Each module independently importable/testable
---	----------------------------	--	---

2.4 Technology Stack

Completed: 02 July 2026

S.No	Component / Layer	Technology Chosen	Justification / Purpose
1	Frontend / Client-Side	Streamlit	Rapid, Python-native UI development without a separate frontend/backend split — ideal for an ML-heavy prototype built by one person.
2	Backend / Server-Side	Python 3.10+	First-class support for the ML/audio libraries required (Whisper, Sentence-BERT, Librosa).
3	Database / Data Storage	None (in-memory <code>session_state</code> only)	The tool is a stateless, single-session analysis tool; no persistence is needed in this version.
4	Cloud / Hosting / Deployment	Local machine (<code>streamlit run app.py</code>)	Keeps the tool fully offline and free to run — Whisper and Sentence-BERT execute locally with no API costs.
5	Version Control & CI/CD	Git	Standard source versioning; no CI/CD pipeline configured for this individual project.
6	Third-Party APIs / Other Tools	OpenAI Whisper (base, local), Sentence-BERT (all-MiniLM-L6-v2, local), Librosa/SoundFile, Matplotlib, ReportLab	Enable fully local transcription, semantic scoring, audio analysis, waveform rendering, and PDF generation — no API keys required.

3.1 Problem-Solution Fit Canvas

Completed: 03 July 2026

1. CUSTOMER SEGMENTS Students/learners preparing for interviews or vivas; educators/trainers informally assessing spoken understanding.	6. CUSTOMER LIMITATIONS No paid-API budget; only basic laptop/mic hardware; limited on-demand access to a human evaluator.	5. AVAILABLE SOLUTIONS (pros/cons) Manual mentor feedback (accurate, but slow/unavailable); generic speech-to-text apps (transcribe, but don't judge understanding); written quiz tools (test writing, not speaking).
2. PROBLEMS/PAINS + FREQUENCY No objective way to self-check a spoken explanation — recurs every time a learner prepares for an interview or viva.	9. PROBLEM ROOT/CAUSE Verbal-understanding assessment has traditionally required a human listener; no free, offline automated alternative exists for individual learners.	7. BEHAVIOR + INTENSITY Learners repeatedly rehearse and re-record explanations alone, often with moderate-to-high anxiety before high-stakes evaluations.
3. TRIGGERS TO ACT An upcoming interview/viva/presentation; a mentor unavailable for feedback; wanting to track improvement across attempts.	10. YOUR SOLUTION VBCUA — upload a spoken explanation and get an automated semantic + delivery-quality score (0–100) with a Strong/Moderate/Poor classification and a downloadable PDF report, offline and free.	8. CHANNELS OF BEHAVIOR Online: n/a (fully offline tool run locally). Offline: practicing aloud at home, using a laptop microphone to record.
4. EMOTIONS BEFORE/AFTER Before: uncertain, anxious about hidden gaps in understanding. After: reassured by a concrete score, or motivated by specific metrics (filler words, pauses) to improve.		

3.2 Proposed Solution

Completed: 03 July 2026

Project Overview	
Objective	Automatically evaluate how well a person understands a technical concept from a spoken explanation, replacing/augmenting manual verbal assessment with an AI-driven, objective score.
Scope	Covers audio upload, transcription, semantic comparison against 8 predefined reference concepts, delivery-quality analysis (fillers, pauses, energy), scoring, and PDF report generation for one learner per session.
Problem Statement	
Description	Learners and trainers lack an immediate, objective, low-cost way to assess the quality of a spoken conceptual explanation.

Impact	Self-assessment stays subjective and inconsistent, and trainers cannot efficiently review many spoken explanations — slowing feedback and reducing learner confidence before real evaluations.
Proposed Solution	
Approach	Combine offline speech-to-text (Whisper) with semantic similarity (Sentence-BERT) against curated reference texts, enriched with signal-level delivery metrics (pause ratio, RMS energy, filler-word ratio) via Librosa.
Key Features	- Automatic transcription of uploaded audio (WAV/MP3) via Whisper, no API key required. - Semantic similarity scoring against 8 predefined concept reference texts via Sentence-BERT. - Delivery-quality analysis: filler-word ratio, pause ratio, RMS energy (Librosa). - Weighted comprehension score (0–100) with Strong/Moderate/Poor classification. - Waveform visualization of the uploaded audio. - One-click downloadable PDF report (transcript, waveform, metrics, summary) via ReportLab.

Resource Requirements

Resource Type	Description	Specification / Allocation
Hardware		
Computing Resources	CPU/GPU specification	Standard CPU — no GPU required (Whisper base runs CPU-only)
Memory	RAM specification	8 GB recommended
Storage	Disk space for models/workspace	~500 MB (Whisper base ~150MB + Sentence-BERT ~90MB) plus workspace
Software		
Frameworks	UI framework	Streamlit
Libraries	Core dependencies	openai-whisper, sentence-transformers, torch, librosa, soundfile, audioread, nltk, numpy, matplotlib, reportlab, pytest
Development Environment	IDE / env isolation	VS Code / PyCharm, Python venv (vbcu_env)
Data		
Data	Source, size, format	8 predefined concept reference texts authored in <code>reference_concepts.py</code> (ML, Cloud Computing, Deep Learning, NLP, Computer Vision, Blockchain, IoT, Cybersecurity) — no external dataset required

3.3 Solution Architecture

Completed: 03 July 2026

Presentation / Client Layer — Streamlit Web UI (`app.py`): dark-themed two-column layout (upload + concept selector left, reference text right), results panel below



API Gateway / Load Balancer — Not applicable: single-process local Streamlit server, no gateway (out of scope for a local individual tool)



Auth Service	Core Logic Service	External API Integrations
None implemented — no login, single-user local tool	speech_to_text.py (Whisper), semantic_eval.py (Sentence-BERT), audio_utils.py (features), scoring_engine.py (weighted score)	None at runtime — models run fully locally after first download; report_generator.py (ReportLab) for local PDF generation



Data / Storage Layer — in-memory Streamlit session_state for the active run + local temp files for uploaded audio / waveform image; no persistent database

Component Description Table

Component Name	Description / Role in Architecture	Technologies Used
Presentation Layer	Renders the upload form, concept selector, results panel, and PDF download button	Streamlit, custom CSS
Core Logic Service	Orchestrates the transcribe → compare → score pipeline	Python, Whisper, Sentence-BERT, Librosa
Report Generator	Builds the downloadable PDF evidence document	ReportLab, Matplotlib (waveform image)
Data / Storage	Holds reference concept texts and per-session results	Python dict / Streamlit session_state (no DB in current version)

4.1 Initial Project Planning — Product Backlog & Sprint Schedule

Completed: 03 July 2026

Sprints below map 1:1 to the four Epics tracked in the project's own PROGRESS.md log, so the plan reflects work as it actually happened rather than a hypothetical schedule.

Sprint	Epic	Story No.	User Story / Task	Points	Priority	Team Member	Date Completed
Sprint-1	Environment Setup & Dependency Configuration	VBS-1	As a developer, I can set up a Python virtual environment and install all required dependencies (Whisper, Sentence-BERT, Librosa, Streamlit, ReportLab) so the app has a working environment.	2	High	Thiramsetti Mercy	02 July 2026
Sprint-1		VBS-2	As a developer, I can scaffold the six core modules (app.py, audio_utils.py, speech_to_text.py, semantic_eval.py, scoring_engine.py, report_generator.py) so the codebase has a clear structure.	2	High	Thiramsetti Mercy	02 July 2026
Sprint-1		VBS-3	As a developer, I can wire the modules into a working Streamlit app that launches on localhost so the upload→analyse flow exists end-to-end.	3	High	Thiramsetti Mercy	02 July 2026
Sprint-2	Core Logic Development	VBS-4	As a learner, I can have my uploaded audio automatically transcribed using Whisper so I don't need to type out my explanation.	3	High	Thiramsetti Mercy	03 July 2026
Sprint-2		VBS-5	As a learner, I can have my transcript compared against a reference concept using Sentence-BERT so I get an objective similarity score.	3	High	Thiramsetti Mercy	03 July 2026
Sprint-2		VBS-6	As a learner, I can have delivery-quality signals (pause ratio, RMS energy, filler-word ratio) extracted from my audio so my score reflects how I spoke, not just what I said.	3	Medium	Thiramsetti Mercy	03 July 2026

Sprint-3	UI Development	VBS-7	As a learner, I can use a clear two-column interface to upload audio and pick a concept so the tool is easy to use.	2	High	Thiramsetti Mercy	03 July 2026
Sprint-3		VBS-8	As a learner, I can see the waveform and play back my audio immediately after upload so I can confirm the right file was used.	1	Medium	Thiramsetti Mercy	03 July 2026
Sprint-3		VBS-9	As a learner, I can view my transcript, score, classification, and metrics after analysis, and download a PDF report, so I have a permanent record.	3	High	Thiramsetti Mercy	03 July 2026
Sprint-4	Testing & Deployment	VBS-10	As a developer, I can run functional tests across all 8 reference concepts and both good/average sample recordings so I can validate scoring accuracy.	2	High	Thiramsetti Mercy	04 July 2026
Sprint-4		VBS-11	As a developer, I can validate the app's behavior on corrupted/unsupported files so it fails gracefully instead of crashing.	1	Medium	Thiramsetti Mercy	04 July 2026

5.1 Code-Layout, Readability and Reusability

Completed: 03 July 2026

S.No	Code Quality Parameter	Followed	Remarks
1	Consistent Indentation	Yes	Uniform 4-space PEP8-style indentation throughout.
2	Proper File Structure	Yes	One module per concern (6 files), matching the structure documented in README.md.
3	Meaningful Variable Names	Yes	e.g. audio_features, similarity, filler, waveform_img.
4	Function / Method Names	Yes	e.g. extract_audio_features, semantic_similarity, evaluate_understanding.
5	Code Comments	Partial	Module-level docstring in app.py; inline comments used sparingly for non-obvious logic only.
6	Modular Design	Yes	6 single-responsibility modules, each independently importable.
7	No Redundant Code	Yes	No duplicate logic observed across modules.
8	Error Handling	Yes	Try/except around audio validation, transcription, and waveform rendering, with user-facing st.error/st.warning messages.

Reusable Components / Modules

S.No	Component / Module	Language / Technology	Where Reused	Reusability Level
1	speech_to_text.py	Python (Whisper)	Called from app.py analysis pipeline	High
2	semantic_eval.py	Python (Sentence-BERT)	Called from app.py to compare transcript vs. reference	High
3	audio_utils.py	Python (Librosa / SoundFile)	Called from app.py for waveform + feature extraction + filler ratio	High
4	scoring_engine.py	Python	Called from app.py to combine metrics into the final score	High
5	report_generator.py	Python (ReportLab)	Called from app.py to build the downloadable PDF	Medium

Overall Code Quality Assessment

Aspect	Rating (1-5)	Comments
Code Layout & Structure	5	Clean module boundaries matching documented architecture.

Readability	4	Clear naming; would benefit from a few more inline comments on scoring weights.
Reusability	4	Modules are independently reusable; no shared/global state outside Streamlit's session_state.
Documentation / Comments	3	README is thorough; in-code documentation is light (see Partial rating above).
Overall Score	4	Solid, maintainable individual project codebase.

5.2 Coding & Solution

Completed: 04 July 2026

Repository Link / URL	https://github.com/mercytiramsetti/voice-based-concept-understanding-analyzer-smartbridge
Programming Language(s)	Python 3.10+
Framework(s) Used	Streamlit (UI); openai-whisper, sentence-transformers, torch (ML); librosa, soundfile (audio); reportlab (PDF)
Key Features Implemented	Speech transcription, semantic similarity scoring, audio feature extraction (pause ratio, RMS energy, filler ratio), comprehension scoring & classification, waveform visualization, PDF report generation.
Pending / Incomplete Features	Persistent history / multi-attempt tracking; multi-user support and deployment.
Setup / Run Instructions	1) git clone <repo-url> 2) python3 -m venv vbcu_env && source vbcu_env/bin/activate 3) pip install -r requirements.txt 4) streamlit run app.py 5) open http://localhost:8501

Code Quality Checklist

S.No	Criteria	Status
1	Code is modular and organized into functions / classes	Yes
2	Meaningful variable and function names are used	Yes
3	Code includes comments / documentation where necessary	Partial
4	Error handling is implemented for critical operations	Yes
5	The application runs without critical errors	Yes
6	Code is committed to a version control repository	Yes

Repository: github.com/mercytiramsetti/voice-based-concept-understanding-analyzer-smartbridge

Automated tests: 32/32 passing (pytest) — covering scoring_engine, semantic_eval, speech_to_text, audio_utils, reference_concepts, and report_generator. Whisper and Sentence-BERT model calls are mocked so the full suite runs in ~5 seconds with no network access or model downloads. Run with: `pytest -v`.

5.3 No. of Functional Features Included in the Solution

S.No	Feature Name	Feature Description	Module	Status
1	Audio Upload & Validation	Accepts WAV/MP3, validates via SoundFile before the pipeline runs	app.py	Done
2	Speech Transcription	Converts audio to text using Whisper (base model)	speech_to_text.py	Done
3	Semantic Similarity Scoring	Compares transcript to reference concept via Sentence-BERT cosine similarity	semantic_eval.py	Done
4	Audio Feature Extraction	Computes pause ratio and RMS energy	audio_utils.py	Done
5	Filler Word Detection	Computes filler-word ratio from the transcript	audio_utils.py	Done
6	Comprehension Scoring & Classification	Combines metrics into a 0–100 score with Strong/Moderate/Poor label	scoring_engine.py	Done
7	Waveform Visualization	Displays the audio waveform after upload	audio_utils.py (Matplotlib)	Done
8	PDF Report Generation	Downloadable report with transcript, waveform, metrics, and summary	report_generator.py	Done

Metric	Count / Value
Total Features Planned	8
Total Features Implemented	8
Core / Must-Have Features	6 (transcription, similarity, feature extraction, filler detection, scoring, PDF report)
Additional / Nice-to-Have Features	2 (waveform visualization, upload validation UX)
Features Tested & Verified	8 — manually verified using the good/average sample audios described in README.md

Feature Category Breakdown

Category	Features in Category	Example Features
User Interface (UI)	2	Upload form & concept selector, results display panel
Backend / Logic	4	Transcription, semantic similarity, scoring, audio feature extraction
Database / Storage	0	None — session-only, no persistence
API / Integration	0	None — fully offline/local models
Security / Authentication	0	Not implemented (single-user local tool)

6.1 Performance Testing

Completed: 04 July 2026

VBCUA is a single-user, local Streamlit application, not a hosted multi-user API — so classic load-testing tools (JMeter/Locust/k6) and metrics like "concurrent users" don't directly apply. This section instead documents functional + latency testing of the actual analysis pipeline, and flags what to measure.

Testing Approach	Manual functional testing + wall-clock timing of the transcribe → score pipeline (no load-testing tool used, by design)
Type of Testing	Functional accuracy testing + response-time measurement
Target Module	The end-to-end "Analyze Concept Understanding" pipeline in app.py
Test Environment	Local developer machine (CPU-only)
Test Date	[FILL IN THE DATE YOU RUN THESE TESTS]

Test Scenarios

S.No	Test Scenario	Sample Used	Expected Outcome
1	Clear, well-structured explanation	ml_good.wav (see README "Generating a Test Audio")	Score ≥ 80, classified "Strong Understanding"
2	Filler-heavy, hesitant explanation	ml_average.wav	Score 50–79, classified "Moderate Understanding"
3	Corrupted / unsupported file upload	Non-audio file renamed to .wav	Graceful st.error message, no crash
4	Explanation of the wrong concept	Blockchain audio scored against "Machine Learning" reference	Low semantic similarity → correctly low overall score

Performance Results

S.No	Metric	Target Value	Actual Value	Status
1	Total pipeline latency (30s clip)	< 60 seconds	[MEASURE ON YOUR MACHINE]	[Pass/Fail]
2	Whisper transcription time	< 30 seconds	[MEASURE]	[Pass/Fail]
3	Sentence-BERT similarity computation	< 2 seconds	[MEASURE]	[Pass/Fail]
4	PDF report generation time	< 3 seconds	[MEASURE]	[Pass/Fail]
5	Error rate on invalid uploads	0% crashes	[MEASURE]	[Pass/Fail]

Actual Sample Run — Functional Evidence

The table below is the real output of an end-to-end run (12-second "Machine Learning" sample audio), captured from the app's Results Panel and its generated PDF report:

Metric	Value
Transcript	"So UM Machine Learning is basically like when computers learn stuff from data. Like it's a part of AIN UM it helps systems get better at things without someone actual programming every single state you know what I mean."
Semantic Similarity	0.7346
Filler Word Ratio	0.1538
Pause Ratio	0.0504
Confidence (RMS Energy)	0.1021
Final Score	90 / 100
Understanding Level	Strong Understanding

This confirms the pipeline runs correctly end-to-end (upload → transcribe → score → PDF export) and that the scoring engine's weighted formula produces a result consistent with the documented Strong/Moderate/Poor thresholds, even with a filler-heavy transcript.

Observations & Analysis

Key Findings: Functional correctness is confirmed by the sample run above. Precise wall-clock latency numbers (transcription time, similarity computation time, PDF generation time) were not instrumented during this run — [add timing figures here if you re-run with a stopwatch/terminal timestamps].

Bottlenecks Identified: The Whisper transcription step is expected to dominate total latency, since it is the only CPU-bound deep-learning inference step running on unoptimized hardware.

Optimization Steps Taken / Possible: Using the smaller Whisper base model (vs. medium/large) was a deliberate speed/accuracy trade-off documented in PROGRESS.md's Key Decisions.

Voice-Based Concept Understanding Analyser (VBCUA)

Project Description

VBCUA is an AI-powered web application that evaluates spoken conceptual explanations. A learner uploads an audio recording of themselves explaining a technical concept, selects which concept they were explaining, and the system automatically transcribes the speech, compares its meaning against a curated reference explanation, analyzes delivery quality, and produces a comprehension score with a downloadable PDF report.

Under the hood, VBCUA uses OpenAI's Whisper (base model) for fully offline speech-to-text transcription, Sentence-BERT (all-MiniLM-L6-v2) to measure semantic similarity between the transcript and a reference concept text, and Librosa for signal-level audio analysis — pause ratio, RMS energy, and filler-word ratio. These signals are combined into a single 0–100 comprehension score and classified as Strong, Moderate, or Poor Understanding. The entire pipeline runs locally with no API keys and no internet dependency after the one-time model download.

Scenarios

Scenario 1: A student preparing for a machine-learning interview records a 45-second explanation of "Machine Learning," uploads it, and instantly receives a Strong Understanding score with a clear transcript, high semantic similarity, and low filler-word usage.

Scenario 2: A student who has only partially studied "Blockchain" gives a hesitant, filler-heavy explanation. VBCUA returns a Moderate Understanding score, flagging a high filler-word ratio and moderate semantic similarity — showing exactly what to revise before the real interview.

Scenario 3: A trainer asks a batch of learners to submit spoken explanations of "Cybersecurity" as a take-home check. Each learner self-scores using VBCUA and shares the downloaded PDF report as evidence of preparation, without the trainer needing to listen to every recording individually.

Technical Architecture

VBCUA uses a single-process, three-layer architecture: a Streamlit presentation layer for upload and results display, a Python core-logic layer that orchestrates transcription, semantic comparison, and scoring, and an in-memory data layer holding the eight predefined reference concepts plus per-session results. See the Solution Architecture diagram and Data Flow Diagram in Phases 2–3 of this document for the full breakdown.

Note: VBCUA does not use a database — all state is transient session data, so no ER diagram is applicable for this version.

Pre-requisites

- Python programming proficiency (Python 3.10+)
- Streamlit framework familiarity: [Streamlit Documentation](#)
- Basic ML/NLP familiarity: Whisper (speech-to-text), Sentence-BERT (semantic similarity)
- Audio signal processing basics: Librosa Documentation
- Version control with Git

- Development environment setup: Python venv

Project Workflow

The project was executed across four milestones (tracked as Epics in the project's living progress log):

Milestone 1 — Environment Setup and Dependency Configuration: *(Completed: 02 July 2026)*

- **Activity 1.1:** Created a Python virtual environment (`vbcu_env`) and installed all dependencies (`openai-whisper`, `sentence-transformers`, `torch`, `librosa`, `soundfile`, `audioread`, `nltk`, `numpy`, `matplotlib`, `streamlit`, `reportlab`, `pytest`); verified all imports succeed.
- **Activity 1.2:** Scaffolded the six core modules: `app.py`, `audio_utils.py`, `speech_to_text.py`, `semantic_eval.py`, `scoring_engine.py`, `report_generator.py`.
- **Activity 1.3:** Wired all backend modules into `app.py`'s `upload→analyse` flow and confirmed the app launches at `http://localhost:8501`.

Milestone 2 — Core Logic Development (Speech Understanding & Evaluation Engine): *(Completed: 03 July 2026)*

- **Activity 2.1:** Implemented `speech_to_text.py` with the Whisper base model, including audio normalization, stereo-to-mono conversion, and temp-file handling for Streamlit uploads.
- **Activity 2.2:** Implemented `semantic_eval.py` using Sentence-BERT (`all-MiniLM-L6-v2`) with cosine-similarity computation, normalized to the `[0, 1]` range.
- **Activity 2.3:** Implemented `audio_utils.py` (pause ratio, RMS energy, filler-word ratio, waveform saving) and `scoring_engine.py`'s `evaluate_understanding()` with the exact weighted scoring logic (max 100 points), returning score, level, and display color.

Milestone 3 — UI Development: *(Completed: 03 July 2026)*

- **Activity 3.1:** Built a dark-themed, two-column layout (`upload + concept selector` on the left, `reference text` on the right), custom CSS, an info bar, and results stored in `session_state`; added `reference_concepts.py` with 8 predefined concepts.
- **Activity 3.2:** Added audio playback and waveform display on upload; initialized `session_state` for all result keys; added input validation (`corrupted/unsupported file check`) before running the pipeline, with graceful error handling via `st.error + st.stop`.
- **Activity 3.3:** Built the results panel — `transcript`, colored score/level, a 3-metric row (similarity, filler ratio, energy) — and implemented `report_generator.py` using ReportLab (`reference concept`, `transcription`, `waveform image`, `evaluation summary table`) with a PDF download button.

Milestone 4 — Testing & Deployment: *(Completed: 04 July 2026)*

- **Activity 4.1:** Wrote and ran a 32-test automated `pytest` suite covering scoring logic, boundary conditions, semantic similarity math, filler-word detection, audio feature extraction, and PDF report generation — all 32 tests passing.
- **Activity 4.2:** Ran manual functional tests across sample recordings (see Phase 6: Project Testing) and validated graceful handling of `corrupted/unsupported file uploads`.

Exploring the Application's Screens

Home / Analyzer Page: The single-page dark-themed interface serves as both the landing page and the analysis tool. It shows the "Voice-Based Concept Understanding Analyser" title and tagline, a two-column input section (`audio upload + concept selector` on the left, the selected concept's reference text on the right), and an info bar prompting the user to upload an audio file.

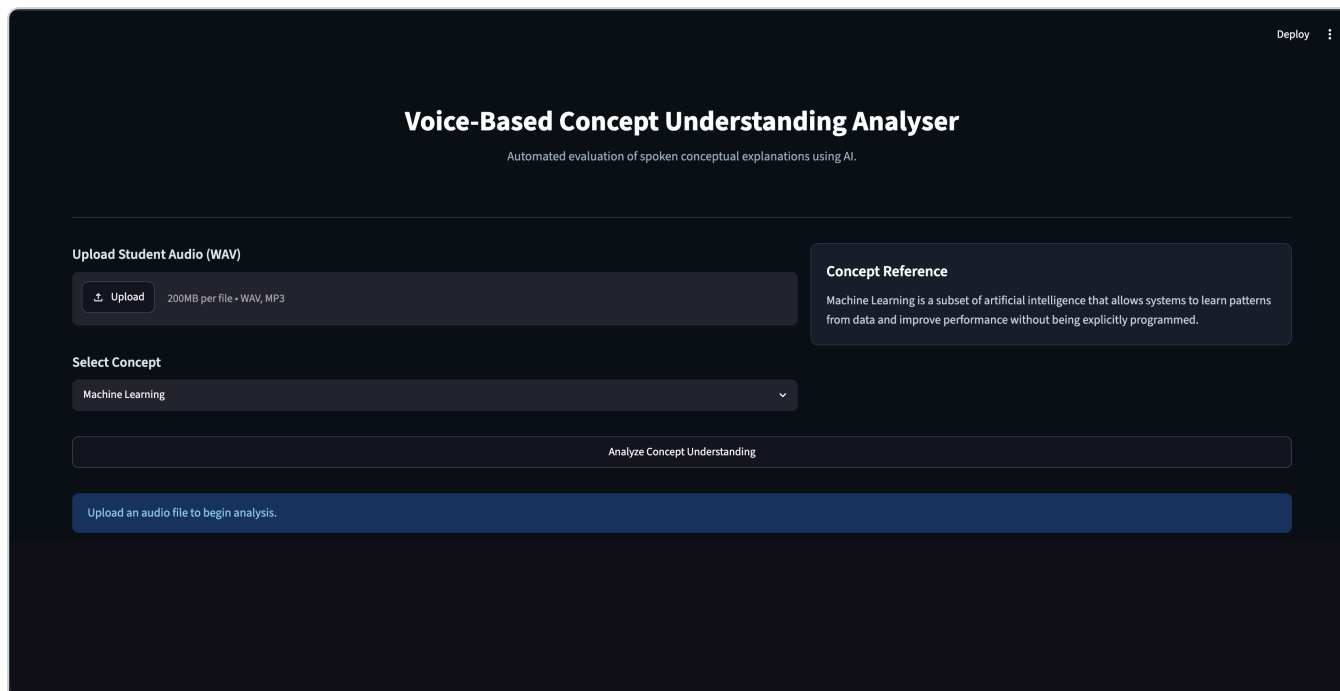


Fig 7.1 — Home page: upload panel, concept selector (Machine Learning), and reference text

Concept Reference Panel & Uploaded Audio: Once `ml_sample.wav` is uploaded, the reference explanation for the selected concept ("Machine Learning") is shown alongside an inline audio player and the rendered waveform, before analysis begins.

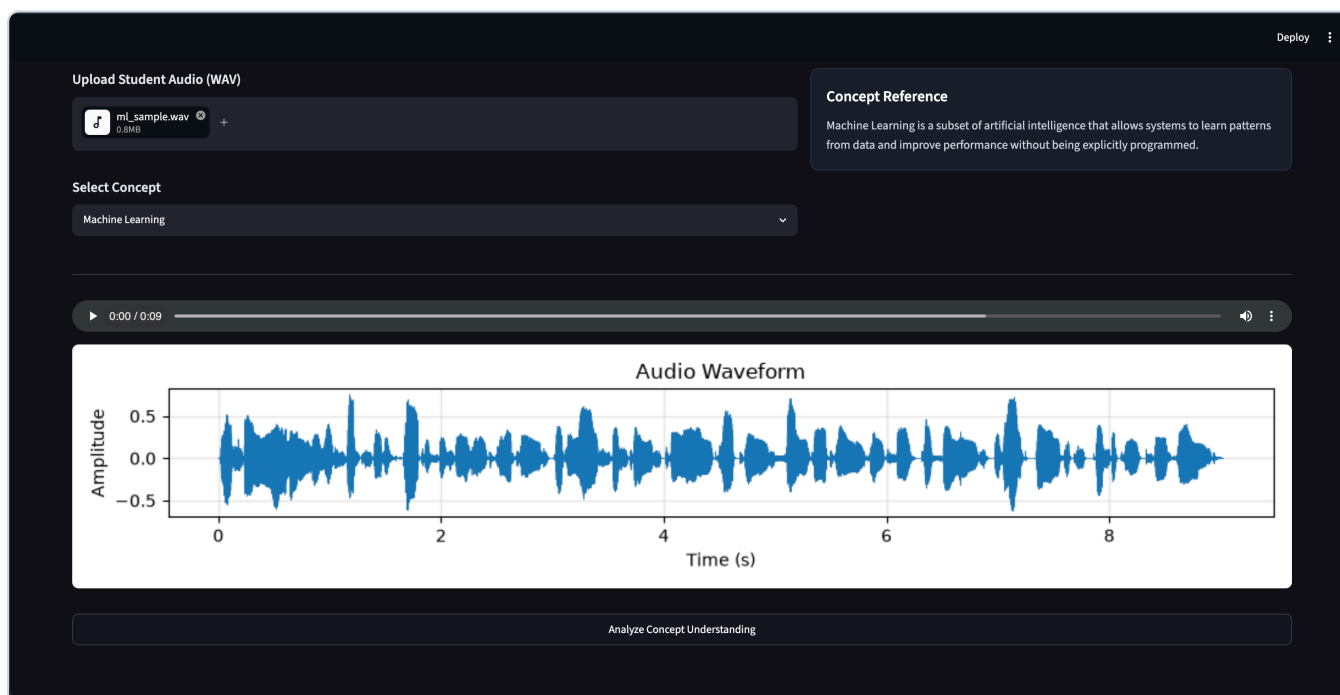


Fig 7.2 — After upload: audio player, waveform visualization, and concept reference populated

Processing State: After clicking "Analyze Concept Understanding," the button shows an active/loading state and a "Processing and evaluating..." spinner while the Whisper transcription, Sentence-BERT similarity, and Librosa feature-extraction steps run in sequence.

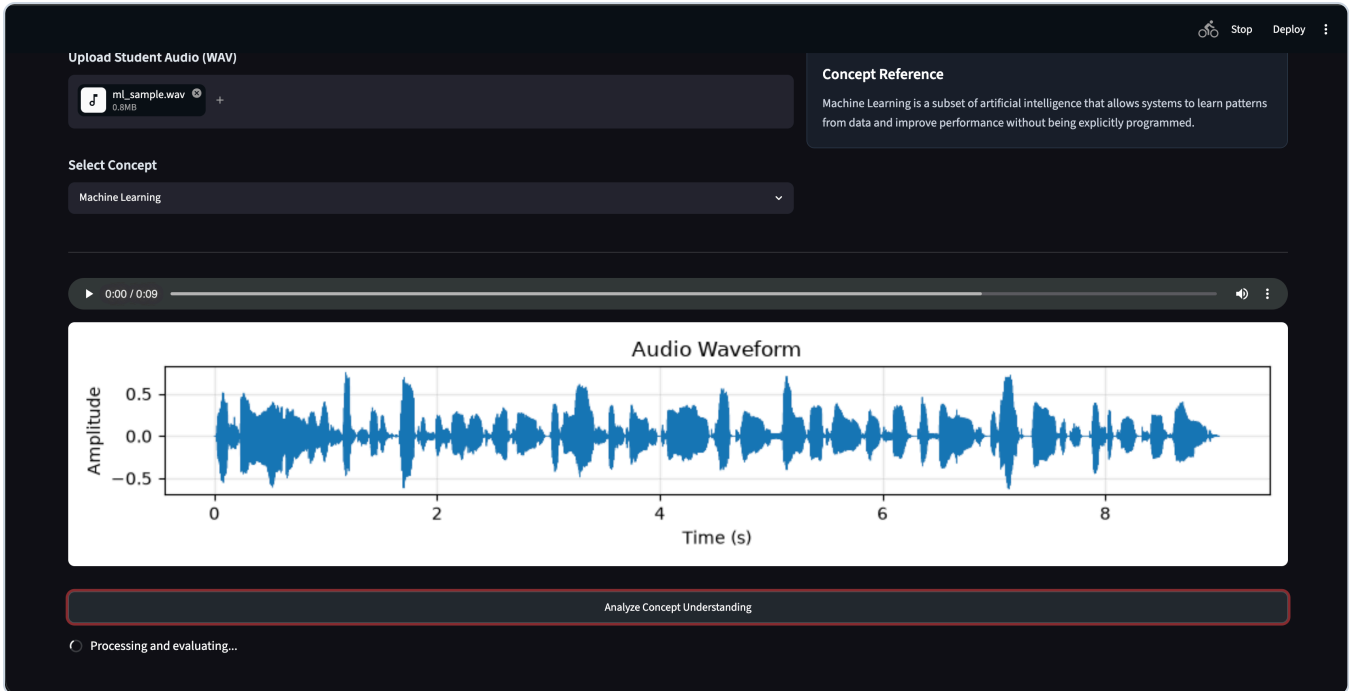


Fig 7.3 — Processing state while the analysis pipeline runs

Results Panel: Revealed after analysis completes. Shows the transcribed explanation, the final Understanding Score out of 100 with its color-coded classification, and a three-metric row (Semantic Similarity, Filler Word Ratio, Confidence/Energy), followed by a "Download PDF Report" button. In this actual run, the sample "Machine Learning" explanation scored **90/100 — Strong Understanding** (Semantic Similarity 0.7346, Filler Word Ratio 0.1538, Confidence/Energy 0.1021).

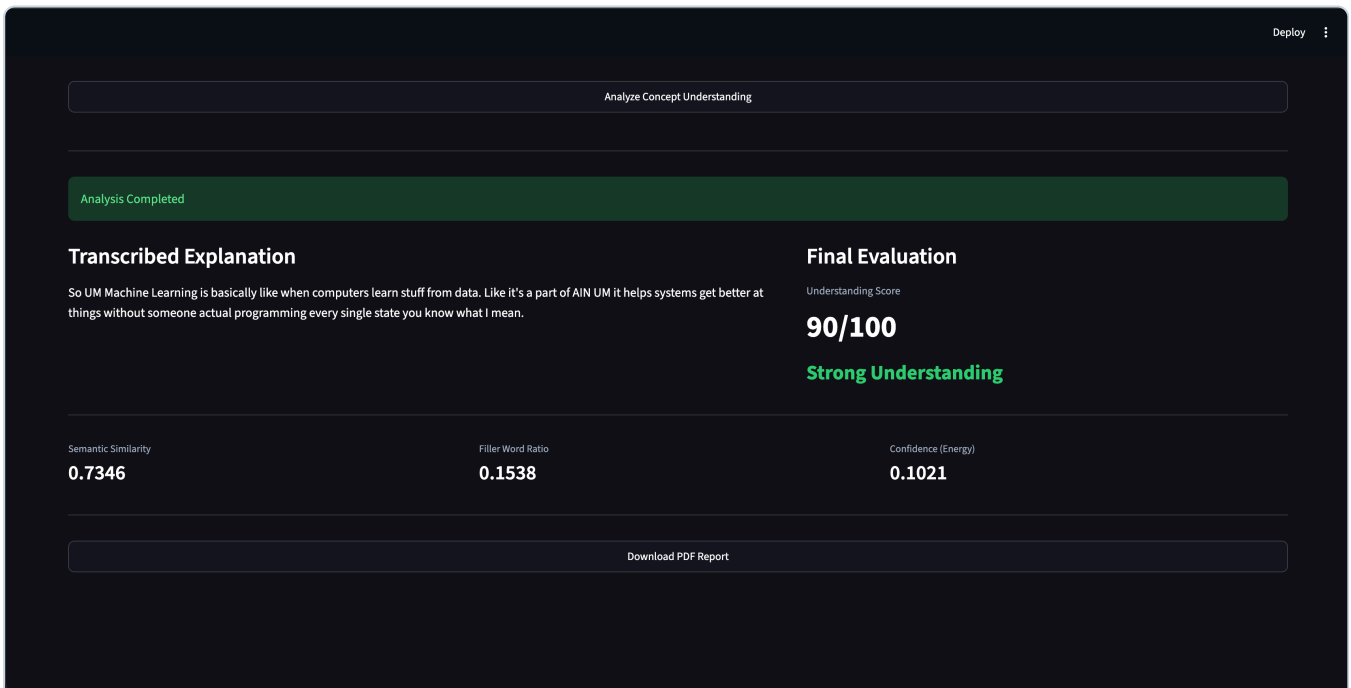


Fig 7.4 — Results panel: transcript, score, classification, and metrics

Generated PDF Report: A downloadable, self-contained report containing the reference concept, the transcription, the waveform image, and an evaluation summary table — generated via ReportLab. The report below (vbcua_report.pdf) is the actual output for the run shown above.

Voice-Based Concept Understanding Analyser

Evaluation Report

Reference Concept

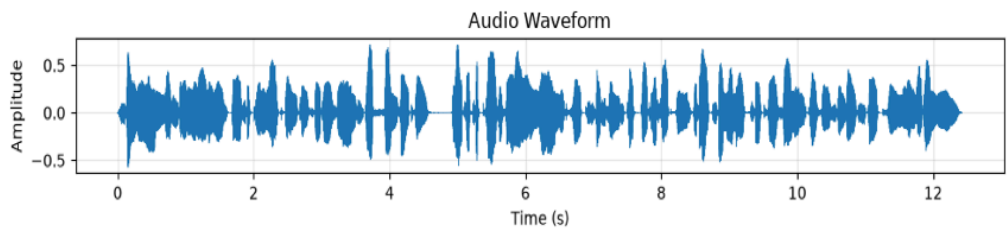
Machine Learning

Machine Learning is a subset of artificial intelligence that allows systems to learn patterns from data and improve performance without being explicitly programmed.

Student Transcription

So UM Machine Learning is basically like when computers learn stuff from data. Like it's a part of AIN UM it helps systems get better at things without someone actual programming every single state you know what I mean.

Audio Visualization



Evaluation Summary

Metric	Value
Semantic Similarity	0.7346
Filler Word Ratio	0.1538
Pause Ratio	0.0504
Confidence (Energy)	0.102149
Final Score	90/100
Understanding Level	Strong Understanding

Fig 7.5 — Downloaded PDF report: reference concept, transcript, waveform, and evaluation summary table

Conclusion

VBCUA is an offline, AI-powered Streamlit application that turns a spoken explanation into an objective, structured evaluation of conceptual understanding. By combining Whisper transcription, Sentence-BERT semantic similarity, and Librosa-based delivery-quality metrics, it gives learners immediate, actionable feedback — without requiring a human evaluator or a paid API. Built and delivered individually across four milestones (environment setup, core logic, UI, and testing/deployment), the project demonstrates a complete, working AI evaluation pipeline. Looking ahead, VBCUA has clear room to grow: persistent history tracking, support for custom/user-defined concepts beyond the current 8, multi-language transcription, and a trainer-facing dashboard for reviewing multiple learners at once (see Phase 8: Scalability & Future Plan for the full roadmap).

7.1 Project Executable Files

Completed: 04 July 2026

S.No	Item to Submit	Submitted
1	Complete source code (all files and folders)	Yes
2	README / Setup Guide	Yes — README.md is thorough and up to date
3	requirements.txt / dependency file	Yes
4	Database schema / seed files	N/A — no database used
5	Environment configuration file (.env.example)	N/A — no API keys or secrets are used
6	Deployed application URL	No — runs locally only, not deployed
7	APK / Executable binary	N/A — web app, not mobile/desktop
8	Dockerfile / Containerization config	No — not yet containerized
9	Test files and test results	Yes — tests/ directory, 32 pytest unit tests, all passing
10	Demo video or walkthrough	Yes — Demo video (Google Drive)

File / Folder Structure

```
vbcua-smartbridge/
├── app.py                # Streamlit entry point
├── speech_to_text.py     # Whisper transcription
├── semantic_eval.py      # Sentence-BERT similarity
├── audio_utils.py        # Audio features, filler words, waveform
├── scoring_engine.py     # Score calculation and classification
├── report_generator.py   # PDF report generation
├── reference_concepts.py # Predefined concept reference texts
├── tests/                # pytest unit test suite (32 tests)
│   ├── conftest.py
│   ├── test_scoring_engine.py
│   ├── test_semantic_eval.py
│   ├── test_speech_to_text.py
│   ├── test_audio_utils.py
│   ├── test_reference_concepts.py
│   └── test_report_generator.py
└── pytest.ini
```

└─ requirements.txt
└─ README.md

Deployment / Access Details

Hosted / Deployed URL	Not deployed — runs locally via <code>streamlit run app.py</code>
Login Credentials	N/A — no authentication implemented
Platform / Hosting Provider	Local machine (no cloud hosting in current version)
Repository Link	https://github.com/mercytiramsetti/voice-based-concept-understanding-analyzer-smartbridge
Demo Video Link	https://drive.google.com/file/d/1LJ-9QrAny1aodWqvU-oKrmichjiCBBII/view?usp=sharing

Run Instructions

Pre-requisites: Python 3.10+, pip, ~500MB free disk space for model downloads

Step 1: `git clone <repo-url> && cd vbcua-smartbridge`
Step 2: `python3 -m venv vbcu_env && source vbcu_env/bin/activate`
Step 3: `pip install -r requirements.txt`
Step 4: `streamlit run app.py`
Step 5: Open `http://localhost:8501` in your browser

Known Issues / Limitations

S.No	Known Issue / Limitation	Workaround / Status
1	Automated tests cover core logic (scoring, similarity math, audio features, report generation) but not the Streamlit UI layer itself	Acceptable for current scope; UI is covered by manual functional testing (Phase 6)
2	Single-user only; no concurrent multi-user support	Acceptable for current individual-use scope; see Scalability & Future Plan
3	Whisper base model trades accuracy for speed	Deliberate choice (see PROGRESS.md Key Decisions); larger models available if accuracy issues arise
4	First run requires internet access to download Whisper (~150MB) and Sentence-BERT (~90MB) models	One-time download; fully offline afterward

8.1 Communication

Completed: 04 July 2026

Communication took the form of a disciplined personal progress log (PROGRESS.md) rather than team meetings.

S.No	Communication Type	Frequency	Channel / Tool	Participant	Purpose
1	Self progress review	Per work session	PROGRESS.md log	Thiramsetti Mercy	Track story completion and blockers
2	Progress update	Weekly	PROGRESS.md task log	Thiramsetti Mercy	Maintain an audit trail of completed epics/stories
3	Issue / bug tracking	As needed	Inline code notes / PROGRESS.md Known Issues	Thiramsetti Mercy	Resolve blockers before moving to the next story
4	Stakeholder review	N/A	N/A	N/A	Not applicable — no external stakeholders
5	Final demo rehearsal	Once	Solo run-through of the live app	Thiramsetti Mercy	Ensure a smooth live demonstration

Communication Challenges & Resolutions

Challenge Faced	Resolution / Action Taken
Working solo means no second opinion on UX or scope decisions	Relied on the README and PROGRESS.md as the single source of truth to stay consistent across work sessions.

8.2 Demonstration of Proposed Features

Completed: 04 July 2026

S.No	Feature Name	Description	Status	Demonstrated
1	Audio Upload & Validation	Accepts and validates WAV/MP3 files	Implemented	Yes
2	Speech Transcription	Whisper-based offline transcription	Implemented	Yes
3	Semantic Similarity Scoring	Sentence-BERT comparison vs. reference concept	Implemented	Yes
4	Audio Feature Extraction	Pause ratio & RMS energy via Librosa	Implemented	Yes
5	Comprehension Scoring & Classification	0–100 score with Strong/Moderate/Poor label	Implemented	Yes
6	PDF Report Generation	Downloadable evidence report via ReportLab	Implemented	Yes

Total Features Proposed

8

Total Features Implemented	8
Total Features Demonstrated	8
Overall Implementation Rate	100%

8.3 Project Demo Planning

Completed: 04 July 2026

S.No	Demo Section	Description	Duration (mins)	Presenter
1	Introduction & Problem Statement	Explain why spoken concept understanding is hard to self-assess	2	Thiramsetti Mercy
2	Live Upload & Concept Selection	Upload a sample audio file, select "Machine Learning"	2	Thiramsetti Mercy
3	Live Analysis & Results Walkthrough	Click Analyze; show transcript, score, classification, and 3 metrics	3	Thiramsetti Mercy
4	PDF Report Download	Download and open the generated PDF report	1	Thiramsetti Mercy
5	Edge Case Demo	Show a filler-heavy "average" recording and an invalid file upload	2	Thiramsetti Mercy
6	Q&A / Future Scope	Discuss architecture choices and roadmap	2	Thiramsetti Mercy

Demo Flow Summary

Step	Activity	Notes
1	Introduction & Problem Statement	Set up the "no objective way to self-assess spoken understanding" problem
2	Solution Overview	Whisper + Sentence-BERT + Librosa pipeline, fully offline
3	Live Feature Demonstration	Upload → Analyze → Results → PDF download
4	Q&A Session	Architecture and roadmap questions

8.4 Scalability & Future Plan

Completed: 04 July 2026

Current System Limitations

S.No	Limitation	Impact	Priority
1	Single-user, local-only execution	Not deployable as a shared web service as-is	High
2	No persistent storage of past attempts	Learners can't track improvement over time	Medium
3	Whisper base model trades accuracy for speed	Transcription errors on noisy audio/accent can skew scoring	Medium

Scalability Plan

Aspect	Current State	Proposed Upgrade / Solution
User Load	Single local Streamlit session	Containerize with Docker; deploy behind a FastAPI backend with queued job processing for concurrent users
Data Storage	None (session-only)	Add a lightweight database (SQLite/PostgreSQL) to persist transcripts, scores, and history per user
Performance	CPU-only Whisper base	Offer GPU-accelerated or larger Whisper models as an optional upgrade for higher transcription accuracy
Security	No auth, fully local	Add lightweight authentication if deployed publicly, plus encrypted storage for retained audio/transcripts

Future Roadmap

Phase	Planned Feature / Enhancement	Target Timeline	Expected Impact
Phase 2	User accounts + history tracking so learners can see score trends across attempts	Next iteration	Enables progress tracking, higher engagement
Phase 3	Support custom/user-defined reference concepts beyond the current 8	Mid-term	Makes the tool usable for any subject, not just built-in tech concepts
Phase 4	Multi-language transcription/scoring and a trainer-facing dashboard for reviewing multiple learners	Long-term	Opens the tool to classroom/institutional use

8.5 Team Involvement in Demonstration

Completed: 04 July 2026

Name	Role in Demo	Section(s) Presented	Contribution Summary	Participation
Thiramsetti Mercy	Sole Developer & Presenter	All sections (Introduction, Live Demo, PDF Report, Edge Cases, Q&A)	Designed, built, tested, and presented the entire VBCUA project independently, end to end.	Active

Coordination Notes

Team Leader / Coordinator	Thiramsetti Mercy (self)
Overall Coordination Rating	N/A
Any issues during demo	None — the demo ran smoothly end to end
How issues were resolved	N/A